Week 1, Day 1: HashMap Internals and Arrays

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: HashMap and ConcurrentHashMap | DSA: Arrays

Today's Schedule

- **0:00–0:45** Study HashMap internals (section 1 below).
- **0:45–1:30** Study ConcurrentHashMap and the equals/hashCode contract (sections 2 and 3).
- **1:30–2:30** DSA: solve the 4 array problems (section 5). Spend at most 25 minutes per problem before reading the hint.
- **2:30–3:00** Answer the interview questions aloud (section 4), then write 10 flashcards from your weak answers.
- **Optional (30 min):** write your own simplified HashMap with put/get and resizing.

1. HashMap Internals

Structure

- A HashMap is an array of buckets (the table). Each bucket holds a linked list of nodes, or a red-black tree when the bin gets large.
- Each node stores hash, key, value and next pointer.
- Default capacity is 16 and the load factor is 0.75. Capacity is always a power of two.

How put(key, value) works

1. Compute the hash: hashCode() XOR (hashCode() >>> 16). This spreads the high bits into the low bits so poor hash codes collide less.
2. Compute the bucket index: (n - 1) & hash, where n is the table length. This equals hash mod n for powers of two, but is faster.
3. If the bucket is empty, place a new node there.
4. If not, walk the bin. If a node has the same hash and an equal key (== or equals()), replace its value. Otherwise append a new node.
5. If the number of entries exceeds capacity x load factor, resize.

Treeification (Java 8+)

- When a single bin reaches 8 nodes and the table capacity is at least 64, the list converts to a red-black tree, making worst-case lookup O(log n) instead of O(n).
- If the table is smaller than 64, HashMap resizes instead of treeifying.

- A tree converts back to a list when it shrinks to 6 nodes.

Resizing

- Capacity doubles and entries are redistributed. Because capacity is a power of two, each entry either stays at its index or moves to index + oldCapacity, decided by one bit of its hash.
- Resizing is expensive. If you know the expected size n, create the map with capacity of about n / 0.75 + 1 to avoid repeated resizes.

Other facts that interviewers like

- HashMap allows one null key (stored in bucket 0) and many null values.
- It is not thread-safe. In Java 7, concurrent resizing could create a cyclic list and an infinite loop. In Java 8 the loop is fixed, but concurrent writes can still lose data.
- Iteration order is not guaranteed. Use LinkedHashMap for insertion order and TreeMap for sorted keys.
- Iterators are fail-fast: structural modification during iteration throws ConcurrentModificationException (best effort, not guaranteed).
- Average get/put is O(1). Worst case is O(log n) with treeification, or O(n) in older versions.

2. ConcurrentHashMap Internals

- **Java 7:** the map was split into segments (default 16), each with its own lock, so up to 16 writers could work in parallel.
- **Java 8+:** segments were removed. Inserting into an empty bin uses a lock-free CAS (compare-and-swap). For a non-empty bin it takes a synchronized lock on the first node of that bin only, so writers on different bins never block each other.
- Reads are generally lock-free, using volatile reads.
- It does **not** allow null keys or null values, because a null return from get() would be ambiguous in a concurrent setting (missing key or null value).
- size() is computed from a base counter plus counter cells (similar to LongAdder) to reduce contention. It is an estimate under concurrent updates.
- Iterators are weakly consistent: they never throw ConcurrentModificationException but may or may not reflect updates made after creation.
- Use atomic methods such as putIfAbsent, computeIfAbsent, compute and merge. A separate get-then-put is a race condition even on a ConcurrentHashMap.

Comparison

- **HashMap:** fastest, not thread-safe, allows null.

- **Hashtable:** every method synchronized on the whole map, legacy, no nulls. Avoid.
- **Collections.synchronizedMap:** wraps a map with one lock, so all operations are serialized and iteration needs manual synchronization.
- **ConcurrentHashMap:** fine-grained locking and CAS, best choice for concurrent access.

3. equals() and hashCode() Contract

- If a.equals(b) is true, a.hashCode() must equal b.hashCode().
- If two objects have the same hashCode, they may or may not be equal (collisions are allowed).
- hashCode() must return the same value while the fields used in equals() do not change.
- **Rule:** override both together, and use the same fields in both.
- **Mutable keys are dangerous.** If a key's hashCode changes after insertion, the entry sits in the wrong bucket and get() cannot find it. Prefer immutable keys such as String, Integer and records.
- If you override equals() but not hashCode(), two equal objects can land in different buckets, so lookups fail and duplicates appear in a HashSet.

```
public final class EmployeeId {
    private final String id;
    public EmployeeId(String id) { this.id = id; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof EmployeeId)) return false;
        return id.equals(((EmployeeId) o).id);
    }
    @Override public int hashCode() { return id.hashCode(); }
}
```

4. Interview Questions to Answer Aloud

1. How does HashMap work internally? Walk through put() and get().
2. What happens when two keys have the same hash code?
3. Why is the capacity always a power of two?
4. When does a bin convert to a tree, and why the threshold of 8 and the table size of 64?
5. What is the load factor and what happens if you set it to 1.0 or 0.5?

6. Why does ConcurrentHashMap not allow null keys or values?
7. How did ConcurrentHashMap change between Java 7 and Java 8?
8. Is get() then put() safe on a ConcurrentHashMap? How do you fix it?
9. What goes wrong if you use a mutable object as a HashMap key?
10. HashMap vs Hashtable vs synchronizedMap vs ConcurrentHashMap: when would you use each?
11. How would you design a cache with a HashMap? (Follow-up: LinkedHashMap with removeEldestEntry for LRU.)
12. What is the difference between fail-fast and weakly consistent iterators?

5. DSA: Arrays (4 Problems)

Problem 1: Two Sum

- **Idea:** one pass with a HashMap from value to index. For each number x, check whether target - x is already in the map.
- **Complexity:** O(n) time, O(n) space.
- **Edge cases:** duplicates (such as [3,3] with target 6), negative numbers, the same element not used twice.

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        Integer j = seen.get(target - nums[i]);
        if (j != null) return new int[]{j, i};
        seen.put(nums[i], i);
    }
    return new int[0];
}
```

Problem 2: Best Time to Buy and Sell Stock

- **Idea:** track the minimum price seen so far and the best profit if you sold today.
- **Complexity:** O(n) time, O(1) space.
- **Edge cases:** prices only decreasing (answer 0), a single day.

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE, best = 0;
    for (int p : prices) {
        minPrice = Math.min(minPrice, p);
        best = Math.max(best, p - minPrice);
    }
    return best;
}
```

Problem 3: Contains Duplicate

- **Idea:** add each number to a HashSet and return true on the first repeat. Alternative: sort and compare neighbors for O(1) extra space.
- **Complexity:** O(n) time with a set, O(n log n) with sorting.
- **Follow-up to prepare:** what if the array is too large for memory?

Problem 4: Product of Array Except Self

- **Idea:** without division, build a prefix product pass left to right, then multiply by a running suffix product right to left, writing into the same result array.
- **Complexity:** O(n) time, O(1) extra space (excluding the output).
- **Edge cases:** zeros in the array (the reason division fails), a single negative number.
- **Hint:** result[i] = (product of everything left of i) x (product of everything right of i).

6. End-of-Day Checklist

- [ ] I can explain put() and get() in HashMap without notes.
- [ ] I can explain why equals() and hashCode() must be overridden together.
- [ ] I can state the Java 7 vs Java 8 change in ConcurrentHashMap.
- [ ] I solved all 4 array problems and can state the time and space complexity of each.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Day 2): JVM memory model and garbage collection, plus string problems.